

CE313 — Cloud Computing
Final Project Report

BAYAN AI

An Event-Driven, Serverless Architecture for Multilingual Audio Processing and Synthesis

Project Team

Dara Shikoh Bodla

Reg. No. 2023176

Muhammad Musa

Reg. No. 2023330

Wali Muhammad Khan

Reg. No. 2023753

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology
May 2026

1. Project Abstract & Problem Statement

Bayan AI is a cloud-native, serverless platform engineered to dissolve the language barrier in educational content. The system ingests raw English-language audio lectures and orchestrates a complex, fully automated pipeline of machine-learning models that transcribe, translate, and summarize the content — outputting a synthesized Urdu audio track alongside structured textual insights.

Rather than a conventional monolithic server, Bayan AI is built entirely on AWS Serverless Computing. The result is a system that scales infinitely on demand, incurs zero cost when idle, and processes heavy audio payloads asynchronously without blocking the user interface.

Core Value Proposition

- Language accessibility: English lecture audio → Urdu audio + study guide, fully automated
- Serverless-first: No provisioned servers; pay-per-invocation cost model
- Event-driven: Every step is triggered by the output of the previous one — zero polling from the UI
- Resilient: AWS Step Functions catch and handle failures at every stage of the pipeline

2. System Architecture Overview

Bayan AI is composed of three interlocking layers: a React single-page application (SPA) on the client, an event-driven serverless backend on AWS, and a real-time WebSocket broadcasting system that keeps the two in sync throughout processing.

The table below maps every technology used to its layer and role in the pipeline:

Layer	Technology / Service	Role
Frontend	React + TypeScript + Vite	SPA — upload UI, WebSocket client, progress display
File Storage	AWS S3	Raw audio ingestion & synthesized MP3 output
REST API	AWS API Gateway (REST)	Pre-signed URL generation & CORS enforcement
WebSocket API	AWS API Gateway (WebSocket)	Duplex real-time push from cloud to browser
Orchestration	AWS Step Functions (ASL)	State machine — sequences all microservices
Compute	AWS Lambda (Python 3.9)	10+ single-purpose serverless functions
Transcription	Amazon Transcribe	Speech-to-text on uploaded audio
Translation	Amazon Translate	English-to-Urdu text conversion
NLP Analysis	Amazon Comprehend	Key phrases, entities, study-guide generation
Speech Synthesis	Amazon Polly (Neural)	Urdu text-to-speech MP3 output

Connection Store	Amazon DynamoDB	WebSocket connectionId persistence
IaC / Deployment	AWS SAM + CloudFormation	Full infrastructure-as-code deployment

3. Frontend Architecture

The client interface is a Single Page Application built with React and TypeScript, optimised via the Vite build tool. It acts as the command centre for the entire pipeline, handling direct-to-cloud file transfers and maintaining a live duplex connection with the backend.

3.1 Direct-to-S3 Upload via Pre-Signed URLs

To prevent any application server from bottlenecking under heavy audio uploads, the frontend uses the Pre-Signed URL pattern:

- The React app sends a lightweight GET request to the REST API Gateway.
- The API returns a cryptographically signed S3 URL, valid for 5 minutes only.
- The browser uses a standard PUT request to upload the MP3 binary directly into the S3 bucket — the application server is never in the data path.

3.2 Real-Time State Management via WebSockets

The entire UI state is driven by asynchronous events pushed from the cloud, with no polling required:

- A `useEffect` hook establishes a persistent `wss://` connection to the AWS API Gateway WebSocket endpoint on page load.
- As the backend Step Function executes, the `BroadcasterFunction` pushes JSON payloads (e.g., `{"status": "TRANSLATING"}`) directly to the browser.
- React's `useState` hook instantly triggers re-renders, advancing a visual progress bar that shows exactly which AI model is active — all without a page refresh.

4. Backend Infrastructure & Compute Layer

The entire backend is codified as Infrastructure-as-Code using the AWS Serverless Application Model (SAM). The `template.yaml` defines over a dozen interlocking cloud resources deployed atomically via AWS CloudFormation.

4.1 Event Orchestration: AWS Step Functions

The core orchestration brain is the `BayanStateMachine`, written in Amazon States Language (ASL). It guarantees the strict sequencing of all microservices and provides built-in resilience:

State Machine Design Highlights

Resilience: If the Transcribe step fails or times out, the Step Function catches the error and safely aborts the pipeline — no infinite loops, no orphaned jobs.

Polling loop: Because transcription is asynchronous and takes variable time, the machine uses a Wait state and a Choice state to continuously check TranscribeStatusFunction until the job reports COMPLETED.

Parallel execution: The ComprehendFunction runs in a Parallel branch alongside translation, reducing total pipeline latency.

Broadcast hooks: BroadcasterFunction is triggered at each major state transition to push live progress to the connected browser.

4.2 Microservices: AWS Lambda Functions

All business logic is divided into highly specialised, single-purpose Python 3.9 Lambda functions. Each runs in an isolated container for only the seconds it needs to complete its task:

Lambda Function	Trigger	Responsibility
PreSignedUrlFunction	REST GET /upload-url	Generate secure S3 pre-signed URL with CORS headers
StarterFunction	S3 ObjectCreated event	Extract bucket/key, generate lectureId, launch Step Function
TranscribeLauncherFunction	Step Function task	Submit async Transcribe job pointing to S3 audio
TranscribeStatusFunction	Step Function poll loop	Check Transcribe job status; extract transcript text on completion
TranslateFunction	Step Function task	Send English text to Amazon Translate (en → ur)
ComprehendFunction	Step Function parallel	Extract key phrases, entities, study guide via Amazon Comprehend
PollyFunction	Step Function task	Synthesize Urdu text to MP3 via Amazon Polly neural voice
FinalizerFunction	Step Function task	Assemble final JSON payload with all outputs
WebSocketManagerFunction	\$connect / \$disconnect	Save / delete connectionId in DynamoDB
BroadcasterFunction	Step Function broadcasts	Query DynamoDB; push progress events via post_to_connection

4.3 WebSocket Broadcasting System

Because Lambda functions are stateless and API Gateway WebSocket connections are ephemeral, the system uses DynamoDB as a connection registry:

- WebSocketManagerFunction intercepts every \$connect event and stores the unique connectionId in DynamoDB. On \$disconnect, it removes it.
- BroadcasterFunction queries DynamoDB for all active connectionIds and uses the API Gateway Management API (post_to_connection) to push JSON status updates to every live browser.
- This architecture cleanly decouples the processing pipeline from client connectivity — any number of concurrent clients can monitor progress independently.

5. Security & IAM Design

The project strictly adheres to the Principle of Least Privilege. There are no global administrator keys in any backend code. All access is scoped at the individual Lambda level:

- **Execution Roles:** Every Lambda function has a uniquely generated IAM Role — functions cannot access resources outside their designated scope.
- **Scoped Policies:** For example, TranscribeLauncherFunction is granted only s3:GetObject on the raw audio bucket and transcribe:StartTranscriptionJob. It has no access to DynamoDB or Amazon Polly.
- **API Security:** The REST API enforces CORS (Cross-Origin Resource Sharing) to ensure only the Bayan AI frontend domain can request upload URLs.
- **Time-limited credentials:** Pre-signed S3 URLs expire after 5 minutes, minimising the window for credential misuse.

6. Deployment Configuration & Presentation Mode

Note on Current Deployment State

AWS Service Control Policies (SCPs) on newly created and educational accounts enforce a 24-hour verification lock that prevents usage of high-tier ML APIs (SubscriptionRequiredException). To demonstrate structural integrity, the system currently operates in Presentation Mode.

The following components are fully live and processing real data:

- **Event-Driven Architecture:** React → S3 → Step Functions → DynamoDB → WebSockets — 100% active
- All Lambda functions are deployed, invoked, and returning data through the full state machine
- WebSocket broadcasting is live: status updates arrive in the browser in real time

The following components are in simulated mode:

- TranscribeLauncherFunction and TranslateFunction return hardcoded JSON payloads that mirror exact production API responses, demonstrating the full data flow and pipeline sequencing without requiring live ML API access.

This approach proves that the distributed microservice pipeline, state machine routing, IAM boundaries, and duplex WebSocket communications all function correctly — simulating the exact latency and data structures of the third-party AI APIs.

7. Future Scope & Scalability

Because the system is entirely serverless, it inherently scales to handle thousands of concurrent audio uploads without any code changes. Planned future enhancements include:

- **Add AWS Cognito for student authentication, linking specific lecture IDs to user accounts and enabling per-user lecture history:** Cognito Integration
- **Save final JSON outputs to a DynamoDB table so students can view a searchable history of all processed lectures:** Persistent Storage
- **Add a language dropdown in the React UI to dynamically pass different target language codes (Arabic, Pashto, etc.) through the Step Function to Amazon Translate and Polly:** Multi-Language Support
- **Aggregate Comprehend outputs across lectures to surface trending topics and concept frequency for instructors:** Analytics Dashboard
- **Expose the pipeline as a REST API for institutional partners to integrate Bayan AI into their own LMS platforms:** Cognito + API Keys

8. Conclusion

Bayan AI demonstrates that a production-grade, AI-powered language platform can be built entirely on serverless primitives — without provisioning a single server. The architecture is event-driven from the moment a student uploads a file to the moment the synthesized Urdu audio lands back in their browser. Every component — from the React SPA and pre-signed URL upload flow, through the Step Functions state machine and specialised Lambda microservices, to the DynamoDB-backed WebSocket broadcaster — is independently scalable, fault-tolerant, and zero-cost at rest.

The project validates the feasibility of democratising educational content through cloud-native ML orchestration, and establishes a solid foundation for the multi-language, multi-user platform described in the future scope.

Project Team

Dara Shikoh Bodla 2023176	Muhammad Musa 2023330	Wali Muhammad Khan 2023753
Ghulam Ishaq Khan Institute of Engineering Sciences and Technology CE313 Cloud Computing May 2026		